

## 3.1.2 Tuples (Ordered, Immutable)

ٹیپل (Tuple) گول بریکٹ () میں لکھا جاتا ہے، ترتیب رکھتا ہے مگر تبدیل نہیں ہو سکتا (Immutable)۔

CS XII

Chapter 3: Programming Fundamentals

Lecture 03 of 18

★ Golden

3.1.2 Tuples (Ordered, Immutable) ★ Golden

LEARN IT

A **tuple** is an ordered collection that is **immutable** — once created, you cannot add, remove or replace items. Think of it as a **read-only list**. Tuples use **parentheses** `()`. Like lists, they allow duplicates, mixed types, and indexing from 0 (including negative indices).

**List []**  
Can add / remove / replace

**Tuple ()**  
LOCKED — read-only

A list is an open box you can refill. A tuple is a sealed box you may only look into.

```
t = (10, 20, 30, 40, 50)
print(t[0])
print(t[-1])
print(t[1:4])
print(t[::-1])
```

```
OUTPUT
10
50
(20, 30, 40)
(50, 40, 30, 20, 10)
```

Slicing a tuple returns a **new tuple**, not a list. Negative indexing and `[::-1]` work exactly as they do for lists.

Immutability

You cannot assign to an index of a tuple. Python raises `TypeError`.

```
t = (1, 2, 3)
t[0] = 99
```

```
OUTPUT
Traceback (most recent call last):
  File "tuple.py", line 2, in <module>
    t[0] = 99
TypeError: 'tuple' object does not support item assignment
```

**Exception:** if a tuple contains a **mutable object** (a list inside the tuple), that inner object's contents can still change. The tuple still holds the same two list objects; you changed a list, not the tuple's structure.

```
t = ((1, 2), [3, 4])
t[0].append(99)
print(t)
```

```
OUTPUT
[(1, 2, 99), [3, 4]]
```

`t[0] = [9, 9]` would still be illegal, because that would replace the tuple's first item. `t[0].append(99)` is legal because it mutates the list that already lives at index 0.

Traversal and methods

Traverse a tuple with `for`, just as you do a list. You are only reading, which is the usual reason to choose a tuple.

```
student = ("Ali", 20, "Computer Science")
for item in student:
    print(item)
```

```
OUTPUT
Ali
20
Computer Science
```

Because a tuple cannot grow or shrink, it has only **two** methods:

| Method                     | Meaning                           |
|----------------------------|-----------------------------------|
| <code>.count(value)</code> | How many times the value appears. |
| <code>.index(value)</code> | Index of the first occurrence.    |

```
t = (10, 20, 10, 30, 10)
print(t.count(10))
print(t.index(20))
```

```
OUTPUT
3
1
```

Advantages of tuples

- **Protection** — coordinates, identity-card numbers or settings cannot be changed by accident.
- **Speed** — a fixed-size tuple is processed slightly faster than a list.
- **Hashable keys** — a tuple of immutables can be used as a dictionary key; a list cannot. (You will use dictionaries in 3.1.4.)

WORKED EXAMPLE

**When to choose a tuple.** A student's roll number, name and class form a record that should not be edited in place:

```
record = ("XII-CS-07", "Ayesha", "Computer Science")
print(record[0])
print(len(record))
```

```
OUTPUT
XII-CS-07
3
```

If later you need to change the class, you build a **new** tuple. You do not assign to `record[2]`.

REMEMBER IT

**Tuple = locked lunchbox.** You can see what is inside; you cannot swap the sandwich. Memory: "Tuple is Tight (immutable), written with round brackets that look like a hug holding the data still."

COMMON MISTAKE

**One-item trap:** `(5)` is just the integer 5 with extra parentheses. A one-item tuple needs a trailing comma: `(5,)`.

```
t1 = (5)
t2 = (5,)
print(type(t1), t1)
print(type(t2), t2)
```

```
OUTPUT
<class 'int'> 5
<class 'tuple'> (5,)
```

An empty tuple is `()`.

اردو میں سمجھیں

ٹیپل (Tuple) گول بریکٹ () میں لکھا جاتا ہے، ترتیب رکھتا ہے مگر تبدیل نہیں ہو سکتا (Immutable). انڈیکس اور سلاش فہرست جیسی ہیں، لیکن `t[0] = 99` سے `TypeError` آتا ہے۔ صرف دو میٹھڈ ہیں: `.count()` اور `.index()`۔ حساس ڈیٹا کے لیے ٹیپل محفوظ ہے۔ اگر ٹیپل کے اندر فہرست ہو تو اس فہرست کے اندر کی اشیاء بدل سکتی ہیں۔

EXAM TIP

Expect "Differentiate list and tuple" (at least four points: brackets, mutability, methods, speed/protection), "Why use a tuple?", and "What happens if you write `t[0] = 1`?" Quote the `TypeError` wording. Mention the inner-list exception if the question is a 6-mark "explain with example." A one-item tuple `(5,)` is a favourite trick question.

CHECK YOURSELF

1. Name the two tuple methods.
2. What error does `t[0] = 99` raise for a tuple?
3. After `t = ((1, 2), [3, 4]); t[0].append(99)`, what is `t`?

Answers: 1. `count` and `index`. 2. `TypeError: 'tuple' object does not support item assignment`. 3. `[(1, 2, 99), [3, 4]]`.

---

Next lecture: 3.1.3 Sets (Unordered, Unique, Math Operations)